

# Dealer Management System

Inventory, CRM, sales deals, service, documents and reports for a car dealership group, in one app

## IN ONE SENTENCE

One system for a dealership's stock, leads, deals, service workshop, documents and reports, with a login for each role.

## BUILT WITH

Node 22, Fastify 5, PostgreSQL 16, React 19, TanStack Query, Vite, Docker

## What this guide covers

- 1 What the project is
- 2 The business problem it solves
- 3 The client's requirements
- 4 How the solution is designed
- 5 How to build it from scratch, step by step
- 6 Testing, deployment and known limits

# Contents

|                                         |    |
|-----------------------------------------|----|
| 1. What this project is                 | 3  |
| 2. The business problem                 | 4  |
| 3. Client requirements                  | 5  |
| 4. How the solution is designed         | 7  |
| 5. Build it from scratch, step by step  | 9  |
| 6. Testing, deployment and known limits | 14 |

# 1. What this project is

A Dealer Management System (DMS) is the operating system of a car dealership. This one covers the day-to-day core, Odoo-style: **inventory** with full cost tracking, a **CRM** pipeline for leads, **sales deals** with a pricing worksheet, trade-ins, finance and manager approval, a **service** department with repair and reconditioning orders, **documents**, and **reports**.

It runs on a server with PostgreSQL, so many staff across several branches can use it at the same time, and every figure a salesperson cannot be trusted with (vehicle cost, gross profit) is hidden from them by the server.

| At a glance |                                                                                                                 |
|-------------|-----------------------------------------------------------------------------------------------------------------|
| Type        | Multi-user web application (staff only)                                                                         |
| Users       | Salespeople, service advisors, managers, admins; one or more branches                                           |
| Stack       | Node 22, Fastify 5, PostgreSQL 16, React 19, TanStack Query, Vite, Docker                                       |
| Demo data   | Two branches, six months of stock, leads, deals and service                                                     |
| Tests       | 7 deal/service maths tests, 21 API tests on a real database, 7 web tests, browser smoke test of each role's day |

## Who uses it

| User            | What they do with it                                                                                           |
|-----------------|----------------------------------------------------------------------------------------------------------------|
| Salesperson     | Works leads, keeps customer records, builds and submits deals, closes approved ones. Never sees cost or gross. |
| Service advisor | Opens repair orders for customer cars, adds labour and parts, issues service invoices.                         |
| Manager         | Buys and prices stock, records costs, appraises trade-ins, approves thin deals, reads reports.                 |
| Admin           | Sets tax, fees, minimum gross, F&I products and labour rate; branches, users and the activity log.             |

## 2. The business problem

Many independent and small franchise dealers run on spreadsheets, a paper deal jacket and an off-the-shelf DMS that is expensive and rigid. Costs of reconditioning are not attached to cars, so nobody knows the real margin; salespeople can discount below cost without a manager noticing; the same car can be promised to two customers; and month-end reporting takes days.

The client, a dealership group with two branches, wants **one system where stock, customers, deals and service are connected**, with approvals that cannot be bypassed and numbers the accountant can trust.

| Pain point today                                              | What it costs the business           | How this project solves it                                                                                                 |
|---------------------------------------------------------------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Reconditioning and transport costs not tracked per car</b> | Real margin unknown; losses hidden   | Costs recorded per car (purchase, recon, transport) and service recon orders post their cost onto the car                  |
| <b>Salespeople can discount too far</b>                       | Deals sold at a loss                 | Deals under the minimum gross must be approved by a manager; salespeople never see cost                                    |
| <b>The same car promised twice</b>                            | Angry customers, lost deals          | A car can be held by one active deal only; the database refuses a second                                                   |
| <b>Leads followed up from memory</b>                          | Leads go cold                        | Pipeline board with owners, follow-up dates and overdue warnings, full timeline                                            |
| <b>Paperwork in folders</b>                                   | Lost IDs and titles, compliance risk | Documents attached to customers, cars and deals, checked by content                                                        |
| <b>Month-end by hand</b>                                      | Days of work; errors                 | Reports for units, front/back gross, salespeople, branches, sources, stock age; CSV export matching the report to the cent |

### Value to the business

- Every car's true margin is visible to management; slow stock is highlighted before it loses value.
- Deals are consistent and auditable: gapless invoice numbers, frozen history, approvals recorded.
- Sales, service and management work from the same customer and vehicle records.

### 3. Client requirements

The brief below is written the way a client would hand it to a developer: what the system must do (functional requirements), how well it must do it (non-functional requirements), and what was explicitly left out. Every item below is implemented in the repository; section 6 explains how the project is tested.

#### Functional requirements

| ID           | Area      | Requirement                                                                                                                                                                                                                       |
|--------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FR-01</b> | Inventory | Stock per branch with purchase cost, reconditioning and transport costs, list price, margin and days in stock.                                                                                                                    |
| <b>FR-02</b> | Inventory | Aging view (30 / 60 / 90+ days) and CSV export.                                                                                                                                                                                   |
| <b>FR-03</b> | CRM       | Leads pipeline board (New → Contacted → Appointment → Negotiation, then won or lost) with source, assigned salesperson and follow-up date; overdue warnings.                                                                      |
| <b>FR-04</b> | CRM       | Activity timeline of calls, emails, visits and test drives.                                                                                                                                                                       |
| <b>FR-05</b> | CRM       | One customer record with their leads, cars bought, service history and files.                                                                                                                                                     |
| <b>FR-06</b> | Deals     | Pricing worksheet: selling price, discount, F&I products, trade-in allowance with loan payoff, tax (with or without trade-in credit), documentation fee, deposit and finance with the monthly payment; totals update as you type. |
| <b>FR-07</b> | Deals     | Deals below the minimum gross go to a manager to approve or send back with a reason.                                                                                                                                              |
| <b>FR-08</b> | Deals     | Closing marks the car sold, adds the trade-in to stock at its appraised value, marks the lead won and issues a gapless invoice number; printable bill of sale.                                                                    |
| <b>FR-09</b> | Service   | Repair orders for customer cars with labour and parts lines, tax and a service invoice.                                                                                                                                           |
| <b>FR-10</b> | Service   | Reconditioning orders for stock cars post their cost onto the car (no tax).                                                                                                                                                       |
| <b>FR-11</b> | Documents | Attach ID, licences, titles, invoices and photos (PDF, JPEG, PNG, WebP up to 5 MB) to customers, cars and deals; downloads are always attachments.                                                                                |
| <b>FR-12</b> | Reports   | Monthly units and front/back gross, gross per car, salesperson and branch results, lead-source conversion, oldest stock, service revenue, and a deals CSV for Excel/Power BI.                                                     |
| <b>FR-13</b> | Admin     | Settings (tax, fees, minimum gross, F&I products, labour rate), branches, users with roles, activity log.                                                                                                                         |

#### Non-functional requirements

| ID            | Quality  | Requirement                                                                                                                   |
|---------------|----------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>NFR-01</b> | Security | Passwords hashed with bcrypt; accounts lock after 5 failures; only hashes of session tokens are stored; sessions expire.      |
| <b>NFR-02</b> | Security | Every change needs a CSRF token and a same-site Origin; strict security headers (Content-Security-Policy, HSTS behind https). |
| <b>NFR-03</b> | Security | Every input validated on the server (zod); all SQL parameterised; rate limits on the API and tighter ones on login.           |
| <b>NFR-04</b> | Security | An activity (audit) log of logins and every business action, viewable by admins.                                              |

| ID            | Quality    | Requirement                                                                                                                                                                    |
|---------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NFR-05</b> | Security   | Salespeople never receive cost or gross from the API, and cannot influence add-on costs or trade-in values (they come from the product list and the manager's appraisal).      |
| <b>NFR-06</b> | Integrity  | A car can be sold only once, even if two salespeople submit at the same moment.                                                                                                |
| <b>NFR-07</b> | Integrity  | Invoice numbers are gapless even when four deals close at once (a counters table updated inside the closing transaction).                                                      |
| <b>NFR-08</b> | Integrity  | A closed deal keeps its numbers when tax or fees change later; closed service orders can't change; two people editing one record get a conflict, not an overwrite.             |
| <b>NFR-09</b> | Integrity  | Money is stored as whole cents.                                                                                                                                                |
| <b>NFR-10</b> | Files      | Uploaded files are checked by content (a renamed program is refused), stored with safe names and served only as downloads.                                                     |
| <b>NFR-11</b> | Operations | Runs as one Docker image next to PostgreSQL; health checks <code>/healthz</code> and <code>/readyz</code> ; database migrations run on start-up safely with several instances. |
| <b>NFR-12</b> | Operations | Structured JSON logs with a request id (returned in the <code>x-request-id</code> header); cookies and tokens are redacted.                                                    |

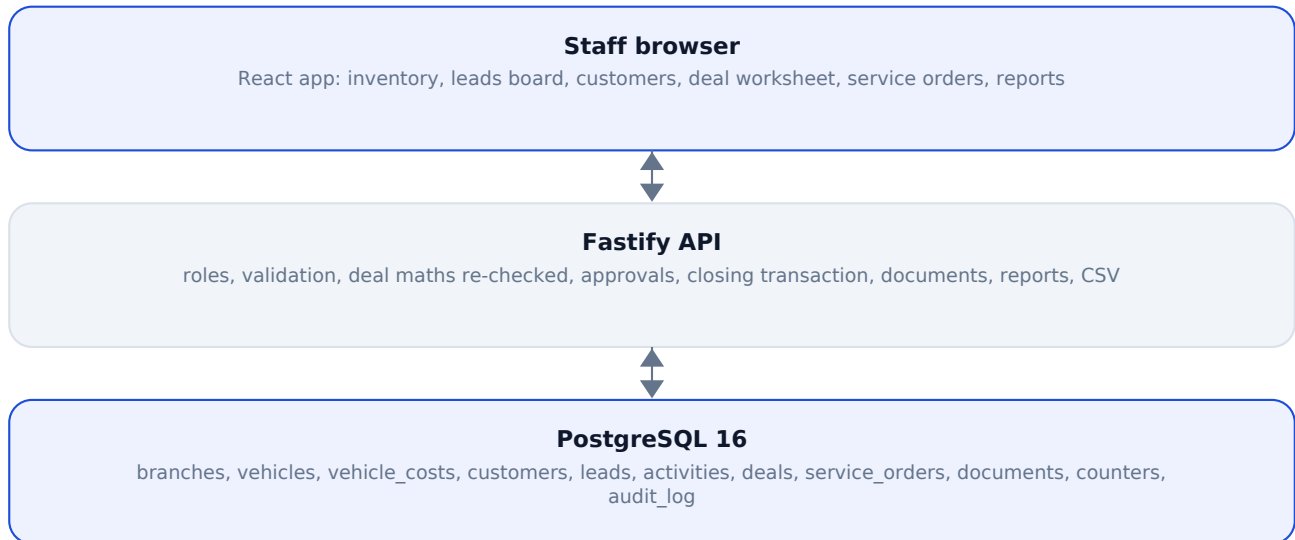
## Out of scope for this release

- General ledger, VAT returns and bank reconciliation (export to the accounting package instead).
- Lender credit applications and title/registration paperwork.
- Unwinding a sold deal; parts stock for the service department.

## 4. How the solution is designed

A classic three-tier web app. The deal and service maths live in `shared/` so the browser shows live totals and the server recalculates the same numbers from trusted inputs before saving.

### Architecture



### Technology choices

| Part                | Choice                                                          | Why                                                                                     |
|---------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Server</b>       | Node 22 + Fastify 5                                             | TypeScript end to end; plugins for security headers, cookies, rate limits, uploads.     |
| <b>Database</b>     | PostgreSQL 16                                                   | Row locks and constraints make "sold once" and gapless numbers reliable.                |
| <b>Shared maths</b> | <code>shared/deal.ts</code> ,<br><code>shared/service.ts</code> | Monthly payment, deal totals, gross and problems as pure functions used on both sides.  |
| <b>Web app</b>      | React 19, TanStack Query,<br>React Router                       | Worksheet with live totals, boards and tables; print view for documents.                |
| <b>Tests</b>        | Vitest, Playwright                                              | Maths checked against a standard amortisation table; each role's day in a real browser. |

### Data model

| Table / file                          | What it holds                                                                                                                                                                          |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>branches / users / sessions</b>    | Branches, staff with roles and branch, sessions.                                                                                                                                       |
| <b>vehicles / vehicle_costs</b>       | Stock with list price, status, branch and version; cost lines (purchase, recon, transport).                                                                                            |
| <b>customers / leads / activities</b> | Customers; leads with stage, source, owner, follow-up; activity timeline.                                                                                                              |
| <b>deals</b>                          | Worksheet values, status (draft, pending_approval, approved, closed, cancelled), frozen totals and gross, invoice number; a sent-back deal returns to draft with the manager's reason. |
| <b>service_orders</b>                 | Repair or reconditioning orders with labour and parts lines and totals.                                                                                                                |

| Table / file                           | What it holds                                              |
|----------------------------------------|------------------------------------------------------------|
| <b>documents</b>                       | File bytes, safe name, type and the record they belong to. |
| <b>counters / settings / audit_log</b> | Gapless number sequences, business settings, activity log. |

## Key business rules

- **Front gross** = selling price – discount – vehicle cost – trade-in over-allowance (allowance above the appraised value); **back gross** = F&I product price – cost, plus the documentation fee. Deals under the minimum gross need approval.
- **Tax** is charged on (price – discount + documentation fee + add-ons), minus the trade-in allowance when the "trade-in credit" setting is on. **Total** = that + tax – trade-in allowance + loan payoff on the trade-in.
- **Finance payment** uses the standard amortisation formula on the amount financed (total – deposit), APR and term.
- **Closing** happens in one transaction: lock the car, check it is still available, mark it sold, insert the trade-in vehicle, mark the lead won, take the next invoice number, freeze the totals.

## Folder layout

```

shared/  deal.ts · service.ts · schemas.ts · types.ts · csv.ts
server/  src/routes (inventory, crm, customers, deals, service, documents, reports, admin,
  auth)
        migrations/001_init.sql · test/dms.test.ts
web/     src/pages (Dashboard, Inventory, Leads, Customers, Deals, DealPage, Service, Reports,
  Print...)
docs/    OPERATIONS.md · PROJECT-GUIDE.pdf

```



## 5. Build it from scratch, step by step

Build in business order: foundation, stock, customers and leads, then deals (the heart of the system), then service, documents and reports.

### Before you start

- Node.js 22+, PostgreSQL 16, Git.
- Understanding of how a car deal is priced (discount, trade-in, tax, finance) helps; section 4 lists the rules.

### Step 1. Set up the workspace

One repository with a server, a web app and shared code used by both.

- 1.1 Create an npm workspace with `server` and `web` packages and a `shared` folder that both import (types, zod schemas and business rules).

```
mkdir dealer-management-system && cd dealer-management-system && git init
npm init -y # then add "workspaces": ["server", "web"] to package.json
npm create vite@latest web -- --template react-ts
mkdir -p server/src server/migrations shared
npm i -w server fastify @fastify/cookie @fastify/helmet @fastify/rate-limit @fastify/static pg
zod @fastify/multipart
npm i -D -w server typescript tsx vitest esbuild @types/node @types/pg
npm i -w web @tanstack/react-query react-router-dom
```

- 1.2 Turn on TypeScript `strict` everywhere. Add root scripts: `dev`, `build`, `typecheck`, `test`, `migrate`, `seed:demo`, `smoke`.

**Done when:** `npm run typecheck` passes on the empty project.

### Step 2. Configuration, database connection and migrations

- 2.1 `server/src/config.ts`: read environment variables with a zod schema (`DATABASE_URL`, `PUBLIC_URL`, `COOKIE_SECURE`, `TRUST_PROXY`, `SESSION_TTL_HOURS`, `RATE_LIMIT_PER_MIN`, `TIMEZONE`) and refuse to start with a clear message when one is wrong.
- 2.2 `server/src/db.ts`: a `pg` connection pool with a statement timeout, and a `tx(fn)` helper that runs `fn` inside `BEGIN/COMMIT` and rolls back on any error.
- 2.3 `server/src/migrate.ts`: apply `migrations/*.sql` in name order, each in its own transaction, recorded in a `schema_migrations` table, under a PostgreSQL advisory lock so several app servers can start at the same time.

```
createdb app_dev
export DATABASE_URL=postgres://postgres:postgres@localhost:5432/app_dev
npm run migrate
```

**Done when:** Running `npm run migrate` twice applies the migrations once and then reports the database is up to date.

### Step 3. Design the database schema

- 3.1 Create the tables listed in the data model with foreign keys, CHECK constraints on statuses and non-negative money, and a `version` column on records people edit.
- 3.2 Add partial unique indexes so only one live deal can hold a car and a VIN appears once in stock, and a `counters` table for gapless numbers.

```
CREATE UNIQUE INDEX deals_one_live_per_vehicle ON deals (vehicle_id)
  WHERE status IN ('pending_approval', 'approved', 'closed');
CREATE UNIQUE INDEX vehicles_vin_in_stock ON vehicles (vin) WHERE vin <> '' AND status <>
'sold';
UPDATE counters SET value = value + 1 WHERE name = 'invoice' RETURNING value; -- inside the
closing transaction
```

**Done when:** The schema migrates; inserting a second active deal for one car fails.

### Step 4. Logins, roles and request security

The same proven foundation every production app here uses.

- 4.1 **Passwords:** hash with Node's built-in `bcrypt` (memory-hard, no native add-on) and compare in constant time. For an unknown email, verify against a dummy hash so response time doesn't reveal which emails exist. Lock the account for 15 minutes after 5 wrong passwords.
- 4.2 **Sessions:** a random 32-byte token in an `httpOnly`, `SameSite=Lax` cookie (`Secure` in production). Store only the token's SHA-256 with a CSRF token, expiry and last-seen time; expire idle sessions.
- 4.3 **Every request:** a Fastify `onRequest` hook loads the session, refuses writes whose `Origin` is another site, and requires the `x-csrf-token` header on every POST/PUT/DELETE.
- 4.4 **Headers and limits:** Helmet with a strict Content-Security-Policy; `@fastify/rate-limit` keyed by the signed-in session (or IP for visitors), with tighter limits on login.
- 4.5 **Roles:** sales (leads, customers, deals; no cost or gross), service (repair orders, customers), manager (stock, costs, approvals, reports) and admin (settings, branches, users, activity log). A `requireUser(...roles)` pre-handler protects each route; the web app hides what a role can't use, but the server is what enforces it.
- 4.6 **Audit log:** an `audit(...)` helper writes who did what, to which record, from which IP, for every business action and every login.
- 4.7 **CLI:** `create-admin` to make the first admin (password from an environment variable or a hidden prompt).

**Done when:** API tests prove: wrong passwords lock the account, a write without the CSRF token is refused, each role gets 403 on routes it may not use.

### Step 5. Deal and service maths in shared code

- 5.1 Write `monthlyPayment()`, `dealTotals()`, `dealGross()`, `worksheetProblems()` and `serviceTotals()` as pure functions on whole cents.
- 5.2 Unit-test them against a published amortisation table, tax with and without trade-in credit, loan payoff and trade-in over-allowance.

**Done when:** Shared tests pass.

## Step 6. Inventory

- 6.1 API: list with filters and paging, create/edit vehicles (managers), cost lines, status changes, aging buckets, CSV export.
- 6.2 Remove cost, margin and gross fields from every response for the sales role (one mapping function used everywhere).
- 6.3 Web: inventory table, vehicle page with costs panel for managers.

**Done when:** API test: a salesperson's responses contain no cost or gross fields.

## Step 7. Customers and CRM

- 7.1 Customers CRUD with search; leads with stage changes logged as activities; follow-up dates with an overdue flag.
- 7.2 Web: drag-free pipeline board (buttons move between stages), customer page with leads, purchases, service history and documents.

**Done when:** A lead can be created, worked and marked lost with a reason.

## Step 8. Deals, approvals and closing

- 8.1 Create a deal from a lead and a car; save the worksheet; the server recalculates totals from trusted inputs (add-on costs from the product list, trade-in value from the manager's appraisal).
- 8.2 Submit: if gross is below the minimum, status becomes `pending_approval`; managers approve, or send back to draft with a required reason.
- 8.3 Close in one transaction (see rules in section 4), then write the audit entries. Build the printable bill of sale.

**Done when:** Tests: two simultaneous submissions for one car → exactly one holds it; four simultaneous closings get four consecutive invoice numbers.

## Step 9. Service orders

- 9.1 Repair orders with labour (hours × rate) and parts lines, tax per settings, close to issue a service invoice number.
- 9.2 Reconditioning orders for stock cars: no tax, and closing posts the cost onto the car as a cost line.

**Done when:** Closed orders are read-only; recon cost appears on the vehicle.

## Step 10. Documents

- 10.1 Multipart upload up to 5 MB; detect the type from the first bytes (PDF, JPEG, PNG, WebP); build a safe file name; store the bytes.
- 10.2 Download with `content-disposition: attachment` and `x-content-type-options: nosniff`.

**Done when:** A renamed .exe is refused; downloads never open inline.

## Step 11. Reports and dashboard

- 11.1 SQL aggregates by month, salesperson, branch and lead source; oldest stock; service revenue.
- 11.2 Deals CSV with formula-injection protection; a test proves it matches the monthly report to the cent.
- 11.3 Dashboard with today's follow-ups, deals waiting for approval and aging stock.

**Done when:** Report totals equal the CSV totals in tests.

## Step 12. Web app shell

- 12.1 `api/client.ts`: a small `fetch` wrapper that sends the CSRF token on writes, turns error responses into readable messages (with per-field details for forms), and sends the user to sign-in on 401.
- 12.2 `api/auth.ts`: TanStack Query hooks for the current user, sign-in and sign-out; a layout with navigation that shows only what the role may use.
- 12.3 Styles with light and dark mode and a phone layout (tables scroll inside their card, forms stack to one column).

**Done when:** You can sign in, see the navigation for your role and sign out.

## Step 13. Demo data

- 13.1 `seed.ts`: two branches, users for each role, six months of stock, costs, leads, deals (some needing approval) and service orders, guarded by `ALLOW_DEMO_SEED` in production.

**Done when:** The demo shows realistic numbers on every page.

## Step 14. Test it end to end

- 14.1 **Unit tests** (Vitest) for every pure business rule in `shared/`.
- 14.2 **API tests** against a real PostgreSQL database: a global set-up drops and recreates the test database (only if its name contains `test`), migrates it and loads the demo data. Tests sign in as each role and call the real HTTP API.
- 14.3 **Browser smoke test** (`web/scripts/smoke.mjs` with Playwright): build, start the real server on a fresh database, then follow each role's day: a lead becomes a deal, a thin deal is approved by a manager, the car is sold with a trade-in, a repair order is invoiced, and reports and phone layouts are checked. Fail on any script error, and save screenshots.

```
npm run typecheck && npm test
npm run build && npm run smoke
```

**Done when:** All three suites pass locally.

## Step 15. Package, automate and document

- 15.1 A multi-stage `Dockerfile`: build everything, then a slim runtime image with production dependencies only, running as a non-root user, with a `HEALTHCHECK` on `/readyz`.
- 15.2 `docker-compose.yml` with PostgreSQL and the app; `.env.example` explaining every setting.
- 15.3 GitHub Actions: typecheck, all tests with a PostgreSQL service container, build, the browser smoke test, a dependency audit, and a job that builds the Docker image and checks it starts and answers.

15.4 [README.md](#) for users and [docs/OPERATIONS.md](#) for whoever runs it (https, backups, monitoring, upgrades).

**Done when:** CI is green on GitHub and `docker compose up -d --build` gives a working app.

## 6. Testing, deployment and known limits

### How it is tested

- **Money:** tax with and without trade-in credit, loan payoff, finance payments against an amortisation table, front and back gross, trade-in over-allowance.
- **Concurrency:** a car is sold once; gapless numbers under simultaneous closings.
- **Approvals can't be bypassed;** closed deals keep their numbers; stale edits are refused; every role sees only its areas.
- **Documents** checked by content; **reports** add up to the CSV.
- 21 API tests, 7 shared tests, 7 web tests and a browser smoke test of each role's day.

```
npm run typecheck
npm test
npm run build && npm run smoke
```

### Deploying and running it

```
cp .env.example .env                # POSTGRES_PASSWORD, PUBLIC_URL, TIMEZONE
docker compose up -d --build
docker compose exec app node dist/cli/create-admin.js --email you@yourdealer.com --name "Your Name"
# demo: docker compose exec -e ALLOW_DEMO_SEED=1 app node dist/cli/seed-demo.js
#       logins admin@, manager@, sales@, sales2@, service@demo.local · password demo-password-1
```

- After signing in, set tax, documentation fee, minimum gross, F&I products and branches under Settings, then add staff logins.

### Known limits

- No general ledger; deals and service invoices export to CSV for the accounting package.
- No lender or DMV integrations; unwinding a sold deal is not supported.
- All users can see every branch's stock and customers (lists filter by branch).
- Parts are priced per service order, not tracked in stock.

### Sensible next steps

- Publish stock automatically to marketplaces through the dealership-automation project.
- Feed deal data into the dealership financials Power BI kit.
- Parts inventory for the service department; accounting integration (Xero/QuickBooks).